

UNIT-3

CHAPTER-1

STACKS

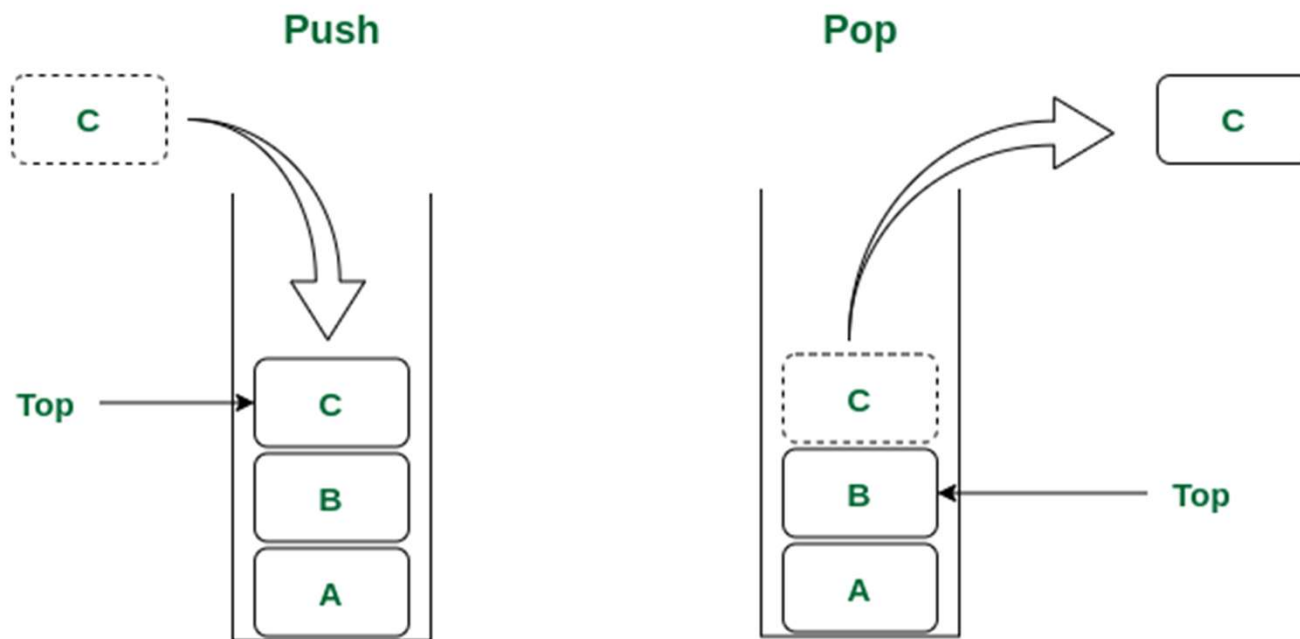
DATA STRUCTURE



Presented By:
Viyola Serrao
Lecturer in BCA Dept

Definition

- A stack is a list of elements in which an element may be inserted or deleted only at one end, called the ***top*** of the stack.
- The elements are removed from a stack in the reverse order of that in which they were inserted into the stack.
- Two operations associated with stacks:
- “**Push**” is the term used to insert an element into a stack.
- “**Pop**” is the term used to delete an element from a stack.



Stack Data Structure

Representation of Stacks

Stacks are represented in memory by means of

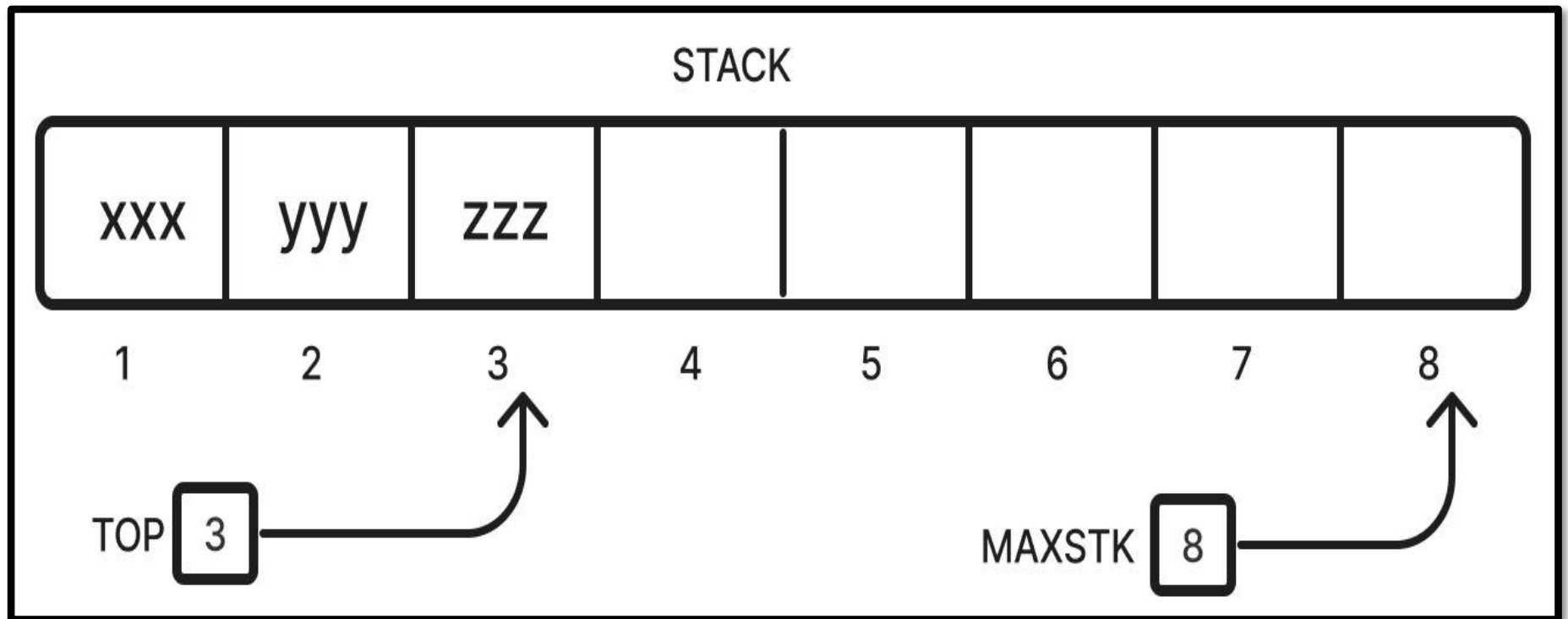
One-Way List

Linear Array

Array Representation of Stacks

- Stacks will be maintained by a linear array **STACK**.
- A pointer variable **TOP**, which contains the location of the top of the element of the stack.
- The variable **MAXSTK**, which gives the maximum number of elements that can be held by the stack.
- The condition **TOP=0** or **TOP=NULL** will indicate that the stack is empty.

Array Representation of Stacks



Array Representation of Stacks

- The operation of adding (pushing) an item onto a stack may be implemented by the procedure, called **PUSH**.
- The operation of removing (popping) an item from a stack may be implemented by the procedures, called **POP**.
- In executing the procedure PUSH, one must first test whether there is room in the stack for the new item;
- If not, then we have the condition known as **overflow**.
- In executing the procedure POP, one must first test whether there is an element in the stack to be deleted;
- If not, then we have the condition known as **underflow**.

Array Representation of Stacks

Procedure 6.1: PUSH(STACK, TOP, MAXSTK, ITEM)

This procedure pushes an ITEM onto a stack.

1. [Stack already filled?]
If $TOP = MAXSTK$, then: Print: OVERFLOW, and Return.
2. Set $TOP := TOP + 1$. [Increases TOP by 1.]
3. Set $STACK[TOP] := ITEM$. [Inserts ITEM in new TOP position.]
4. Return.

Array Representation of Stacks

Procedure 6.2: POP(STACK, TOP, ITEM)

This procedure deletes the top element of STACK and assigns it to the variable ITEM.

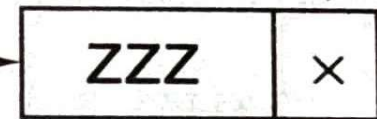
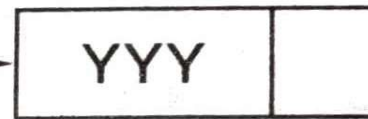
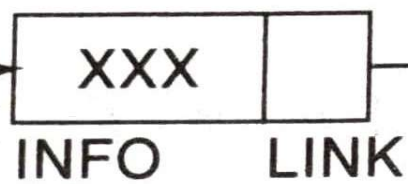
1. [Stack has an item to be removed?]
If $TOP = 0$, then: Print: UNDERFLOW, and Return.
2. Set $ITEM := STACK[TOP]$. [Assigns TOP element to ITEM.]
3. Set $TOP := TOP - 1$. [Decreases TOP by 1.]
4. Return.

Linked Representation of Stacks

- The linked representation of a stack, commonly termed **linked stack** is a stack that is implemented using a singly linked list.
- The **INFO** fields of the nodes hold the elements of the stack.
- The **LINK** fields hold pointers to the neighboring element in the stack.
- The **START** pointer of the linked list behaves as the TOP pointer variable of the stack.
- The **null** pointer of the last node in the list signals the bottom of the stack.

Linked Representation of Stacks

Top (START)



Bottom of stack

Fig. 6.7

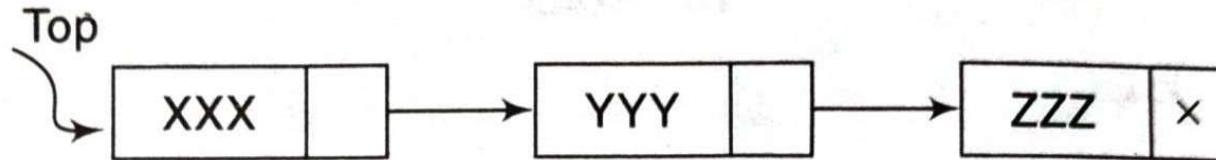
Linked Representation of Stacks

- A **push** operation into STACK is accomplished by inserting a node into the front or start of the list
- A **pop** operation is undertaken by deleting the node pointed to by the START pointer.

Linked Representation of Stacks

Push 'WWW' into STACK

STACK before Push operation:



STACK after Push operation

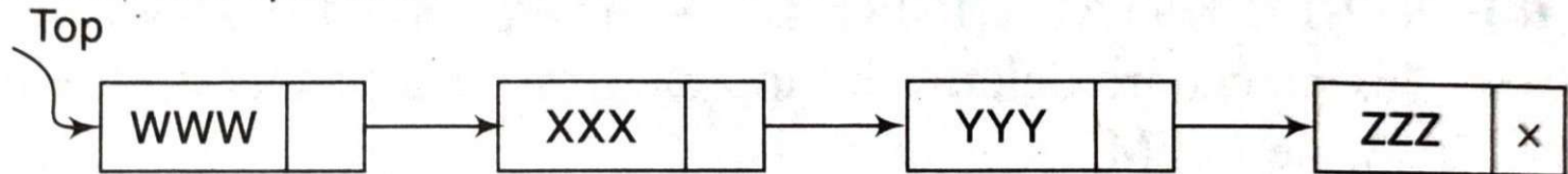
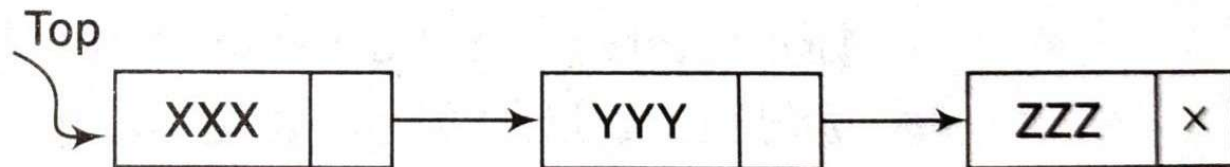


Fig. 6.8

Linked Representation of Stacks

Pop from STACK
STACK before pop operation:



STACK after pop operation

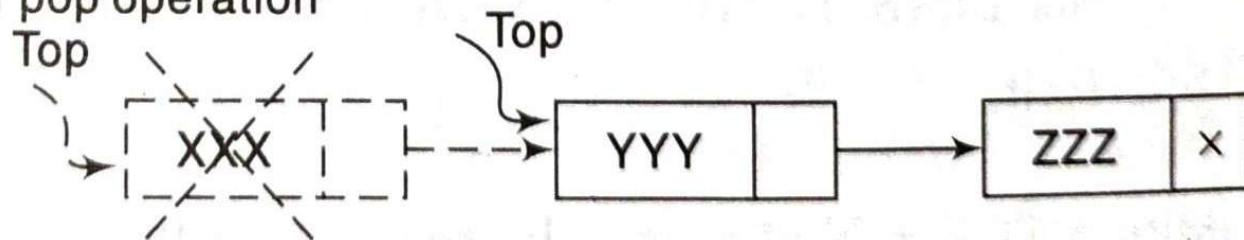


Fig. 6.9

Linked Representation of Stacks

- There is no limitation to pushing items to the linked stack.
- Hence it can support as many push operations (insertion of nodes) as the free-storage list (the AVAIL list) can support.

Linked Representation of Stacks

Procedure 6.3: PUSH_LINKSTACK(INFO, LINK, TOP, AVAIL, ITEM)

This procedure pushes an ITEM into a linked stack

1. [Available space?] If AVAIL = NULL, then Write
OVERFLOW and Exit
2. [Remove first node from AVAIL list]
Set NEW := AVAIL and AVAIL := LINK[AVAIL].
3. Set INFO[NEW] := ITEM [Copies ITEM into new node]
4. Set LINK[NEW] := TOP [New node points to the original top node in
the stack]
5. Set TOP = NEW [Reset TOP to point to the new node at the top of
the stack]
6. Exit.

Linked Representation of Stacks

Procedure 6.4: POP_LINKSTACK(INFO, LINK, TOP, AVAIL, ITEM)

This procedure deletes the top element of a linked stack and assigns it to the variable ITEM

1. [Stack has an item to be removed?]
IF TOP = NULL then Write: UNDERFLOW and Exit.
2. Set ITEM := INFO[TOP] [Copies the top element of stack into ITEM]
3. Set TEMP := TOP and TOP = LINK[TOP]
[Remember the old value of the TOP pointer in TEMP
and reset TOP to point to the next element in the stack]
4. [Return deleted node to the AVAIL list]
Set LINK[TEMP] = AVAIL and AVAIL = TEMP.
5. Exit.

Arithmetic Expression

- Let Q be an arithmetic involving constants and operations.
- The binary operations in Q may have different levels of precedence.
- The following three levels of precedence for the usual five binary operations:

Highest: Exponentiation($\uparrow$)

Next Highest: Multiplication($*$)

Lowest: Addition($+$) and subtraction($-$)

- In any parenthesis-free expression, the operations on the same level are performed from left to right.

Arithmetic Expression

Example 6.5

Suppose we want to evaluate the following parenthesis-free arithmetic expression:

$$2^3 \uparrow 3 + 5 * 2^2 \uparrow 2 - 12 / 6$$

First we evaluate the exponentiations to obtain

$$8 + 5 * 4 - 12 / 6$$

Then we evaluate the multiplication and division to obtain $8 + 20 - 2$. Last, we evaluate the addition and subtraction to obtain the final result, 26. Observe that the expression is traversed three times, each time corresponding to a level of precedence of the operations.

Polish Notation

- In arithmetic operations, if the operator is placed between its two operands is called ***Infix Notations***.
- Example:
- $A + B$ $C - D$ $E * F$ G / H
- ***Polish Notation***, refers to the notation in which the operator symbol is placed before its two operands. This is also called as ***Prefix Notation***.
- $+AB$ $-CD$ $*EF$ $/GH$

Polish Notation

- The following infix expressions into Polish notations using brackets [] to indicate a partial translation:
- $(A+B)*C$
 $= [+AB]*C$
 $= *+ABC$

Polish Notation

- ***Reverse Polish Notation*** refers to the notation in which the operator symbol is placed after its two operands.
- This is also called as ***Postfix Notations***.
- AB+ CD- EF* GH/

Convert infix expression to prefix and postfix form

- a) $A * B + (C \wedge D / E / F)$
- b) $a / b * c - d + e / f * (g + h)$
- c) $(X + Y / Z * W \wedge P) - R$
- d) $(X + Y * Z \wedge A / 4) - P$
- e) $(A + B) * (C - D) / E * F$
- f) $a * b + (c / d \wedge e) - f$
- g) $(A - B * C \wedge D) / (E + F)$
- h) $(a / b) * (c * f + (a - d) * e)$
- i) $A + B * (C - D * (E + F))$
- j) $a * b - c \wedge d + e / f$

Symbol Scanned	STACK	Expression P
A	-	A
*	*	A
B	*	A B
+	+	A B *
(	+ (	A B *
C	+ (	A B * C
^	+ (^	A B * C
D	+ (^	A B * C D
/	+ (/	A B * C D ^
E	+ (/	A B * C D ^ E
/	+ (/	A B * C D ^ E /
F	+ (/	A B * C D ^ E / F
)	+	A B * C D ^ E / F /
END	-	A B * C D ^ E / F / +

Symbol Scanned	STACK	Expression P
a	-	a
/	/	a
b	/	a b
*	*	a b /
c	*	a b / c
-	-	a b / c *
d	-	a b / c * d
+	+	a b / c * d -
e	+	a b / c * d - e
/	+ /	a b / c * d - e
f	+ /	a b / c * d - e f
*	+ *	a b / c * d - e f /
(	+ * (	a b / c * d - e f /
g	+ * (	a b / c * d - e f / g
+	+ * (+	a b / c * d - e f / g
h	+ * (+	a b / c * d - e f / g h
)	+ *	a b / c * d - e f / g h +
END	-	a b / c * d - e f / g h + * +

Symbol	STACK	Expression

X	-	X
+	+	X
Y	+	X Y
/	+ /	X Y
Z	+ /	X Y Z
*	+ *	X Y Z /
W	+ *	X Y Z / W
^	+ * ^	X Y Z / W
P	+ * ^	X Y Z / W P
)	-	X Y Z / W P ^ * +
-	-	X Y Z / W P ^ * +
R	-	X Y Z / W P ^ * + R
END	-	X Y Z / W P ^ * + R -

Symbol	STACK	Expression

X	-	X
+	+	X
Y	+	X Y
*	+ *	X Y
Z	+ *	X Y Z
^	+ * ^	X Y Z
A	+ * ^	X Y Z A
/	+ /	X Y Z A ^ *
4	+ /	X Y Z A ^ * 4
)	-	X Y Z A ^ * 4 / +
-	-	X Y Z A ^ * 4 / +
P	-	X Y Z A ^ * 4 / + P
END	-	X Y Z A ^ * 4 / + P -

Symbol	STACK	Expression

A	-	A
+	+	A
B	+	A B
)	-	A B +
*	*	A B +
C	*	A B + C
-	* -	A B + C
D	* -	A B + C D
)	*	A B + C D -
/	/	A B + C D - *
E	/	A B + C D - * E
*	*	A B + C D - * E /
F	*	A B + C D - * E / F
END	-	A B + C D - * E / F *

4. $a * b + (c / d ^ e) - f$

Symbol	STACK	Expression

a	-	a
*	*	a
b	*	a b
+	+	a b *
c	+	a b * c
/	+ /	a b * c
d	+ /	a b * c d
^	+ / ^	a b * c d
e	+ / ^	a b * c d e
)	+	a b * c d e ^ /
-	-	a b * c d e ^ / +
f	-	a b * c d e ^ / + f
END	-	a b * c d e ^ / + f -

5. (A - B * C ^ D) / (E + F)

Symbol	STACK	Expression

A	-	A
-	-	A
B	-	A B
*	- *	A B
C	- *	A B C
^	- * ^	A B C
D	- * ^	A B C D
)	-	A B C D ^ * -
/	/	A B C D ^ * -
E	/	A B C D ^ * - E
+	/ +	A B C D ^ * - E
F	/ +	A B C D ^ * - E F
)	-	A B C D ^ * - E F + /

6. $(a / b) * (c * f + (a - d) * e)$

Symbol	STACK	Expression

a	-	a
/	/	a
b	/	a b
)	-	a b /
*	*	a b /
c	*	a b / c
*	* *	a b / c
f	* *	a b / c f
+	* +	a b / c f *
a	* +	a b / c f * a
-	* + -	a b / c f * a
d	* + -	a b / c f * a d
)	* +	a b / c f * a d -
*	* *	a b / c f * a d -
e	* *	a b / c f * a d - e
)	*	a b / c f * a d - e * +
END	-	a b / c f * a d - e * +



7. $A + B * (C - D * (E + F))$

Symbol	STACK	Expression

A	-	A
+	+	A
B	+	A B
*	+ *	A B
C	+ *	A B C
-	+ * -	A B C
D	+ * -	A B C D
*	+ * - *	A B C D
E	+ * - *	A B C D E
+	+ * - * +	A B C D E
F	+ * - * +	A B C D E F
)	+ * - *	A B C D E F +
)	+ *	A B C D E F + * -
END	-	A B C D E F + * - * +

8. $a * b - c ^ d + e / f$

Symbol	STACK	Expression

a	-	a
*	*	a
b	*	a b
-	-	a b *
c	-	a b * c
^	- ^	a b * c
d	- ^	a b * c d
+	+	a b * c d ^ -
e	+	a b * c d ^ - e
/	+ /	a b * c d ^ - e
f	+ /	a b * c d ^ - e f
END	-	a b * c d ^ - e f / +

Evaluation of a Postfix Expression

- Suppose P is an arithmetic expression written in postfix notation.
- The following algorithm, which uses a stack to hold operands, evaluates P.

Evaluation of a Postfix Expression

Algorithm 6.5: This algorithm finds the VALUE of an arithmetic expression P written in postfix notation.

1. Add a right parenthesis “)” at the end of P. [This acts as a sentinel.]
 2. Scan P from left to right and repeat Steps 3 and 4 for each element of P until the sentinel “)” is encountered.
 3. If an operand is encountered, put it on STACK.
 4. If an operator $\otimes$ is encountered, then:
 - (a) Remove the two top elements of STACK, where A is the top element and B is the next-to-top element.
 - (b) Evaluate $B \otimes A$.
 - (c) Place the result of (b) back on STACK.[End of If structure.]
- [End of Step 2 loop.]
5. Set VALUE equal to the top element on STACK.
 6. Exit.

Example

- Consider the following arithmetic expression P written in postfix notation:
- P : 5, 6, 2, +, *, 12, 4, /, -
- The equivalent expression Q as follows:
- The parentheses are necessary for the infix expression Q but not for the postfix expression P.
- We evaluate P by using Algorithm 6.5.
- First we add the right parentheses at the end of P to obtain:
- P:5, 6, 2, +, *, 12, 4, /, -,)

Example

- The elements of P have been labeled from left to right for easy reference.
- The contents of STACK as each element of P is scanned.
- The final number in STACK, 37, which is assigned to VALUE when the sentinel “)” is scanned, is the value of P.

Example

<i>Symbol Scanned</i>		<i>STACK</i>
(1)	5	5
(2)	6	5, 6
(3)	2	5, 6, 2
(4)	+	5, 8
(5)	*	40
(6)	12	40, 12
(7)	4	40, 12, 4
(8)	/	40, 3
(9)	-	37
(10)	)	

Fig. 6.11

Evaluate the following postfix expression showing the stack status.

P: 5, 6, 2, +, *, 1, 2, 4, /, ^

P: 3, 5, +, 6, 4, -, *, 4, 1, -, 2, ^, + =25

P: 3, 1, +, 2, ^, 7, 4, -, 2, *, +, 5, - =17

Transforming Infix into Postfix Expression

- Let Q be an arithmetic expression written in infix expression.
- It contains operands and operators along with left and right parentheses.
- Assume that the operators in Q consist of exponentiations($\uparrow$), multiplications($*$), divisions ($/$), additions($+$) and subtractions($-$).
- Operators on the same level, including exponentiation, are performed from left to right unless otherwise indicated by parentheses.

Transforming Infix into Postfix Expression

- The algorithm transforms the infix expression Q into its equivalent postfix expression P.
- The algorithm uses a stack to temporarily hold operators and left parentheses.
- The postfix expression P will be constructed from left to right using the operands from Q and the operators which are removed from STACK.
- We begin by pushing a left parenthesis onto STACK and adding a right parenthesis at the end of Q.
- The algorithm is completed when STACK is empty.

Algorithm 6.6: POLISH(Q, P)

Suppose Q is an arithmetic expression written in infix notation. This algorithm finds the equivalent postfix expression P.

1. Push "(" onto STACK, and add ")" to the end of Q.
2. Scan Q from left to right and repeat Steps 3 to 6 for each element of Q until the STACK is empty:
 3. If an operand is encountered, add it to P.
 4. If a left parenthesis is encountered, push it onto STACK.
 5. If an operator $\otimes$ is encountered, then:
 - (a) Repeatedly pop from STACK and add to P each operator (on the top of STACK) which has the same precedence as or higher precedence than $\otimes$.
 - (b) Add $\otimes$ to STACK.[End of If structure.]
 6. If a right parenthesis is encountered, then:
 - (a) Repeatedly pop from STACK and add to P each operator (on the top of STACK) until a left parenthesis is encountered.
 - (b) Remove the left parenthesis. [Do not add the left parenthesis to P.][End of If structure.][End of Step 2 loop.]
7. Exit.

Transforming Infix into Postfix Expression(Example)

- Consider the following arithmetic infix expression Q:

$$Q: A + (B * C - (D / E \uparrow F) * G) * H$$

Operator Precedence & Associativity:

1. Parentheses $()$ – Highest precedence.
2. Exponentiation $\uparrow$ – Right associative.
3. Multiplication $*$, Division $/$ – Left associative.
4. Addition $+$, Subtraction $-$ – Left associative.

Transforming Infix into Postfix Expression(Example)

Symbol Scanned		STACK	Expression P
(1)	A	(	A
(2)	+	(+	A
(3)	(	(+ (	A
(4)	B	(+ (	A B
(5)	*	(+ (*	A B
(6)	C	(+ (*	A B C
(7)	-	(+ (-	A B C *
(8)	(	(+ (- (	A B C *
(9)	D	(+ (- (	A B C * D
(10)	/	(+ (- (/	A B C * D
(11)	E	(+ (- (/	A B C * D E
(12)	↑	(+ (- (/ ↑	A B C * D E E
(13)	F	(+ (- (/ ↑	A B C * D E E F
(14)	)	(+ (-	A B C * D E E F ↑ /
(15)	*	(+ (- *	A B C * D E E F ↑ /
(16)	G	(+ (- *	A B C * D E E F ↑ / G
(17)	)	(+	A B C * D E E F ↑ / G * -
(18)	*	(+ *	A B C * D E E F ↑ / G * -
(19)	H	(+ *	A B C * D E E F ↑ / G * - H
(20)	)		A B C * D E E F ↑ / G * - H * +

Fig. 6.12

Transforming Infix into Postfix Expression(Example)

- Convert the following Infix expression into Postfix expression using stack status.

Q: $(A + (B * C - (D / E ^ F) * G) * H)$

- Convert the following Infix expression into Postfix expression using stack status.

Q: $((A + B) * D) ^ (E - F)$

Write an algorithm for PUSH and POP operations using arrays.

Algorithm: PUSH(STACK, MAX, TOP, ITEM)

1. Start
2. If `TOP == MAX - 1`
 - Print "Stack Overflow"
 - Go to Step 6
3. Else
 - `TOP = TOP + 1`
 - `STACK[TOP] = ITEM`
4. Print "Item inserted" (optional)
5. End If
6. Stop

Write an algorithm to push and POP operation onto a stack using linked list

Algorithm: POP(STACK, TOP)

1. Start
2. If `TOP == -1`
 - Print "Stack Underflow"
 - Go to Step 6
3. Else
 - `ITEM = STACK[TOP]`
 - `TOP = TOP - 1`
4. Print or return `ITEM`
5. End If
6. Stop

Write an algorithm to find factorial of a number N using recursion.

Algorithm: FACTORIAL(N)

1. Start
2. If `N == 0` or `N == 1`
 - Return `1`
3. Else
 - Return `N * FACTORIAL(N - 1)`
4. Stop

Algorithm to Generate Fibonacci Series using Recursion

Algorithm: FIBONACCI(N)

1. Start
2. If `N == 0`
 - Return `0`
3. Else if `N == 1`
 - Return `1`
4. Else
 - Return `FIBONACCI(N - 1) + FIBONACCI(N - 2)`
5. Stop

Algorithm: Evaluate_Postfix(P)

1. Start
2. Create an empty stack `s`
3. Scan the postfix expression from **left to right**
4. For each symbol `x` in expression:
 - If `x` is an **operand**, push it onto stack `s`
 - If `x` is an **operator**:
 1. Pop operand2 from stack
 2. Pop operand1 from stack
 3. Compute: `result = operand1 operator operand2`
 4. Push `result` back onto stack
5. After scanning all symbols, the final result will be at the **top of the stack**
6. Display the result
7. Stop

Algorithm: Infix_to_Postfix(E)

1. Start
2. Create an empty stack **S**
3. Initialize empty postfix expression **P**
4. Scan the infix expression from **left to right**
5. For each symbol **x**:
 - If **x** is an **operand**, append it to **P**
 - If **x** is **'('**, push it onto stack **S**
 - If **x** is **')'**:
 - Pop from stack and append to **P** until **'('** is found
 - Pop and discard **'('**
 - If **x** is an **operator**:
 - While stack is not empty and precedence of top of stack $\geq$ precedence of **x**:
 - Pop from stack and append to **P**
 - Push **x** onto stack
6. After scanning all symbols, **pop all remaining operators** from stack and append to **P**
7. Output postfix expression **P**
8. Stop



UNIT-3

CHAPTER-2

QUEUES

DATA STRUCTURE



Presented By:
Viyola Serrao
Lecturer in BCA Dept

Define queue. Name the primary queue operations.

- A queue is a **linear list** of elements in which deletions can take place only at one end, called the **front**, and insertions can take place only at the other end, called the **rear**.
- Queues are also called **first-in first-out** (FIFO) lists, since the first element in a queue will be the first element out of the queue.
- In other words, the order in which elements enter a queue is the order in which they leave

Differentiate queue and circular queue.

Feature	Queue (Linear Queue)	Circular Queue
Definition	A linear data structure where insertion happens at rear and deletion at front (FIFO).	A queue in which the last position is connected back to the first, forming a circle .
Memory Utilization	Not efficient – unused spaces at front cannot be reused.	Efficient – empty spaces are reused due to circular nature.
Overflow Condition	Occurs when $\text{rear} == n-1$	Occurs when $(\text{rear} + 1) \% n == \text{front}$
Space Wastage	Possible after deletions	No wastage of space
Structure	Linear (straight line)	Circular (end connects to beginning)
Movement of Pointers	Rear moves only forward	Rear and front move in circular manner using modulo
Implementation Complexity	Simple	Slightly more complex

What is circular queue?

A **Circular Queue** is a type of queue in which the **last position is connected back to the first position**, forming a circular structure.

Define Deque and its Types

Deque (Double Ended Queue) is a linear data structure in which **insertion and deletion can be performed at both ends** (front and rear).

- ◆ **Types of Deque:**

Input-Restricted Deque

Output-Restricted Deque

What do you mean by input-restricted deque and output-restricted deque.

Input-Restricted Deque

Insertion is allowed at **only one end** (usually rear)

Deletion is allowed from **both ends** (front and rear)

👉 Example: You can add elements only from one side but remove from both sides.

◆ Output-Restricted Deque

Insertion is allowed at **both ends**

Deletion is allowed from **only one end** (usually front)

👉 Example: You can insert anywhere, but removal happens from one side only.

What is meant by priority queue? What is its use?

A **Priority Queue** is a special type of queue in which **each element is associated with a priority**, and elements are served based on priority rather than just arrival time.

Write a note on i) circular queue.

ii) Dequeue.

A **circular queue** is a type of linear data structure in which the last position is connected back to the first position, forming a circular structure. It follows the FIFO (First In First Out) principle.

The main purpose of a circular queue is to overcome the limitation of a simple queue, where space at the front cannot be reused once elements are deleted.

In a circular queue, when the rear reaches the last position, it wraps around to the front, allowing efficient utilization of memory. The basic operations of a circular queue include insertion (enqueue) at the rear and deletion (dequeue) from the front. The queue is considered full when the condition $(\text{rear} + 1) \% n == \text{front}$ is satisfied, and empty when $\text{front} == -1$. Circular queues are widely used in applications such as CPU scheduling, buffering, and real-time data processing systems.

A **deque (double ended queue)** is a linear data structure in which insertion and deletion operations can be performed at both ends, namely the front and the rear. This makes it more flexible compared to a simple queue. A deque has two main types: input-restricted deque and output-restricted deque. In an input-restricted deque, insertion is allowed only at one end, while deletion can be performed at both ends. In an output-restricted deque, insertion can be done at both ends, but deletion is restricted to only one end. The main operations of a deque include insertion and deletion at both the front and rear. Due to its flexibility, a deque can be used both as a stack and a queue. It is commonly used in applications such as task scheduling, palindrome checking, and managing undo/redo operations in software.

ALGORITHMS FOR INSERTION & DELETION IN A LINEAR QUEUE DYNAMIC IMPLEMENTATION USING LINKED LIST

(1) Algorithm for inserting an element in a Linear Queue :

Let PTR is the structure pointer which allocates memory for the new node.
element to be inserted into linear queue, INFO represents the information part of
LINK represents the link or next pointer pointing to the address of next
represents the address of first node, REAR represents the address of the last
Before inserting first element in the queue, FRONT=REAR=NULL.

Step 1 : Allocate memory for the new node using PTR.

Step 2 : Read NUM to be inserted into linear queue.

Step 3 : Set PTR->INFO = NUM

Step 4 : Set PTR->LINK= NULL

Step 5 : If FRONT = NULL : then

 Set FRONT=REAR=PTR

Else

 Set REAR->LINK=PTR;

 Set REAR=PTR;

[End of If Else Structure]

Step 6 : Exit

(2) Algorithm for Deleting a node from a Linear Queue :

Let PTR is the structure pointer which deallocates memory of the first node of linear queue & NUM is the element to be deleted from queue, INFO represents information part of the deleted node and LINK represents the link or next pointer of deleted node pointing to the address of next node. FRONT represents the address of first node, REAR represents the address of the last node.

Step 1 : If FRONT = NULL : then

Write 'Queue is Empty(Queue Underflow)' and return.

[End of If structure]

Step 2 : Set PTR = FRONT

Step 3 : Set NUM = PTR->INFO

Step 4 : Write 'Deleted element from linear queue is : ', NUM.

Step 5 : Set FRONT = FRONT->LINK

Step 6 : If FRONT = NULL : then

Set REAR = NULL.

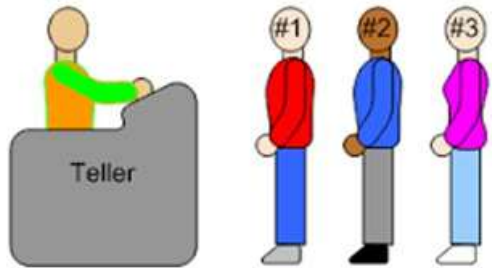
[End of If Structure].

Step 7 : Deallocate memory of the node at the beginning of queue using PTR.

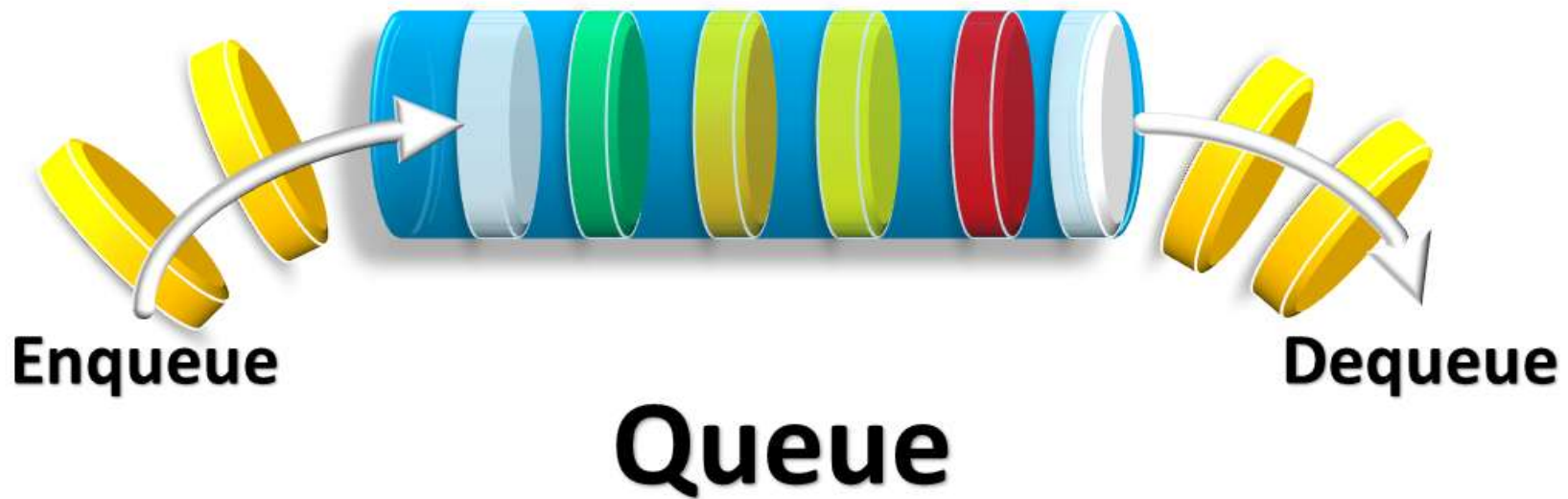
Step 8 : Exit.

Definition

- A queue is a **linear list** of elements in which deletions can take place only at one end, called the **front**, and insertions can take place only at the other end, called the **rear**.
- Queues are also called **first-in first-out** (FIFO) lists, since the first element in a queue will be the first element out of the queue.
- In other words, the order in which elements enter a queue is the order in which they leave.

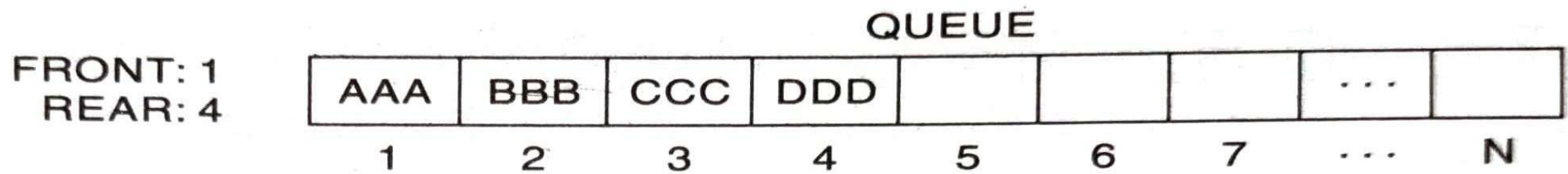


FIFO (First in First Out)

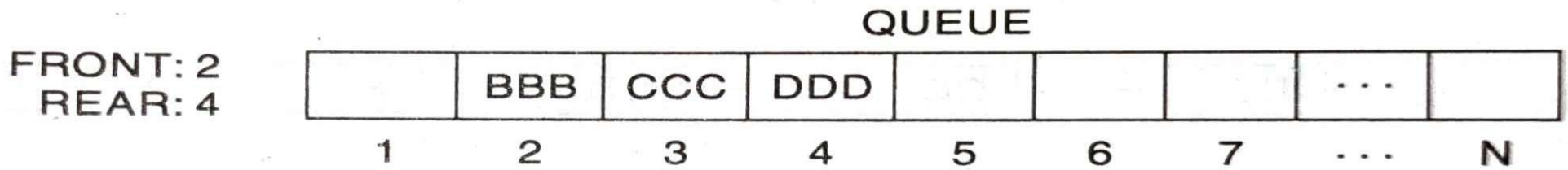


Array Representation of Queue

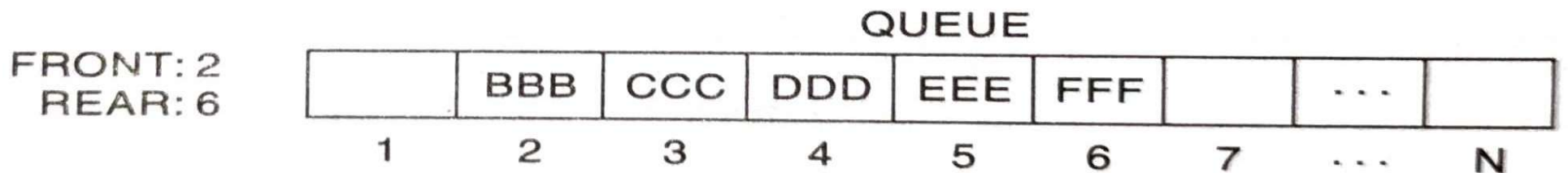
- Queues will be maintained by a linear array **QUEUE** and two pointer variables:
- **FRONT**, containing the location of the front element of the queue;
- **REAR**, containing the location of the rear element of the queue. The condition **FRONT=NULL** will indicate that the queue is empty.



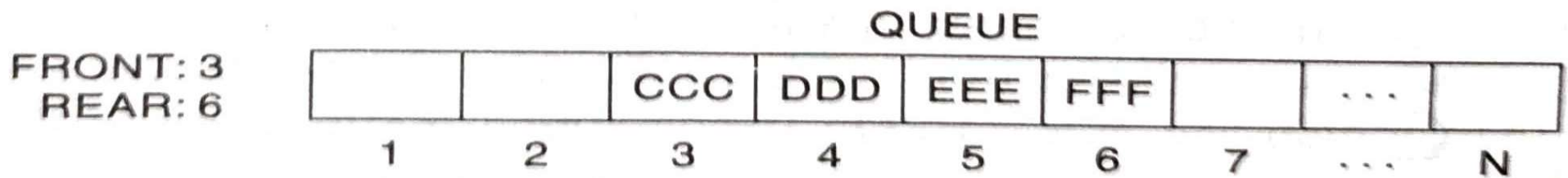
(a)



(b)



(c)



(d)

Fig. 6.20 Array Representation of a Queue

Array Representation of Queue

- Figure 6.20 shows the way that how elements will be stored in memory using an array QUEUE with N elements.
- It also indicates the way elements will be deleted from the queue and the way new elements will be added to the queue.
- Whenever an element is deleted from the queue, the value of FRONT is increased by 1; **FRONT:=FRONT + 1**
- Whenever an element is added to the queue, the value of REAR is increased by 1; **REAR := REAR + 1**

Array Representation of Queue

Procedure: QINSERT(Queue, N, FRONT, REAR, ITEM)

This procedure inserts an element ITEM into a queue.

1. [Queue already filled?]

If $\text{FRONT} = 1$ and $\text{REAR} = N$, or if $\text{FRONT} = \text{REAR} + 1$, then:

Write: OVERFLOW, and Return.

2. [Find new value of Rear.]

If $\text{FRONT} = \text{NULL}$, then: [Queue initially empty.]

Set $\text{FRONT} := 1$ and $\text{REAR} := 1$.

Else if $\text{REAR} = N$, then:

Set $\text{REAR} := 1$

Else:

Set $\text{REAR} := \text{REAR} + 1$

[End of If Structure.]

3. Set $\text{QUEUE}[\text{REAR}] := \text{ITEM}$. [This inserts new element]

4. Return.

Array Representation of Queue

Procedure: QDELETE(Queue, N, FRONT, REAR, ITEM)

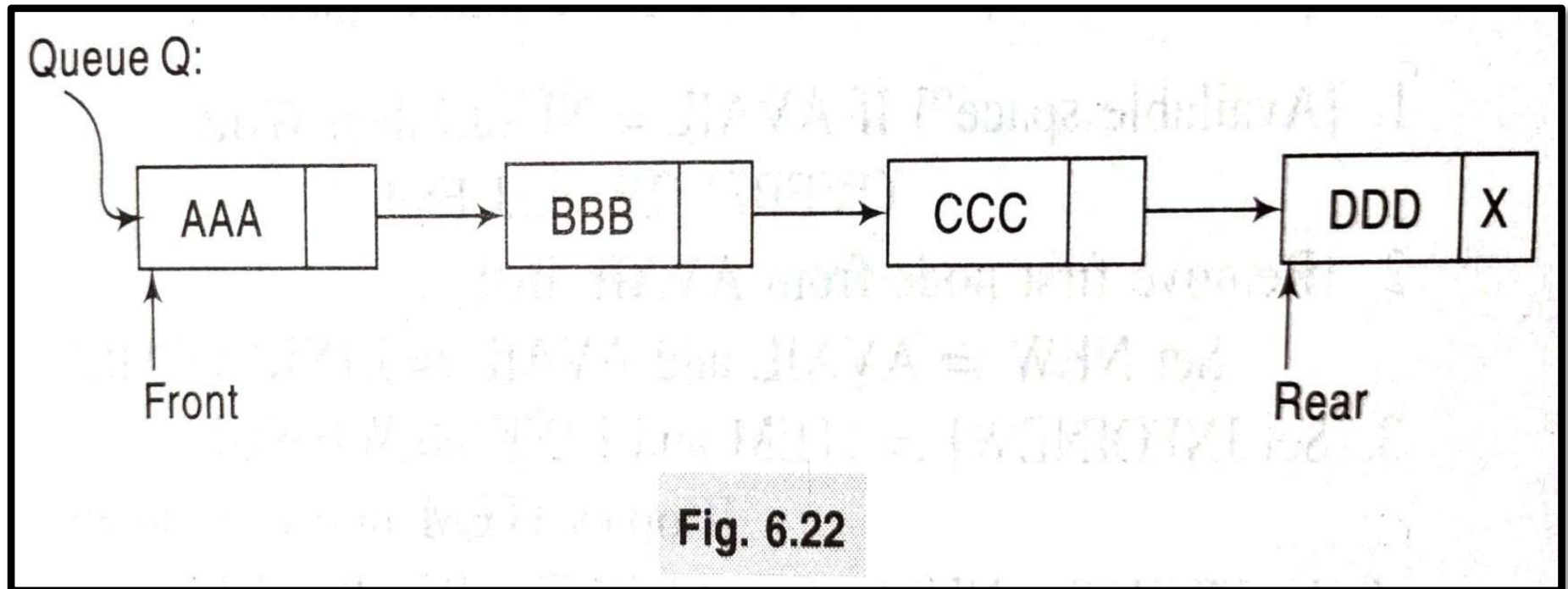
This procedure deletes an element from a queue and assigns it to the variable ITEM.

1. [Queue already empty?]
2. If FRONT = NULL, then: Write: UNDERFLOW and Return.
3. [Find new value of FRONT.]
If FRONT = REAR, then: [Queue has only one element to start.]
 Set FRONT := NULL, and REAR := NULL.
Else if FRONT = N, then:
 Set FRONT := 1.
Else:
 Set FRONT := FRONT + 1.
[End of If Structure.]
4. Return.

Linked Representation of Queues

- A **linked queue** is a queue implemented as a **linked list** with two pointer variables **FRONT** and **REAR** pointing to the nodes which is in the FRONT and REAR of the queue.
- The **INFO** fields of the list hold the elements of the queue and the **LINK** fields hold pointers to the neighbouring elements in the queue.
- In the case of insertion into a linked queue, a node borrowed from the **AVAIL list** and carrying the item to be inserted is added as the last node of the linked list representing the queue.
- The REAR pointer is updated to point to the last node just added to the list.
- In the case of deletion, the first node of the list pointed to by FRONT is deleted and the FRONT pointer is updated to point to the next node in the list.

Linked Representation of Queues



Linked Representation of Queues

Insert 'EEE' into queue Q:

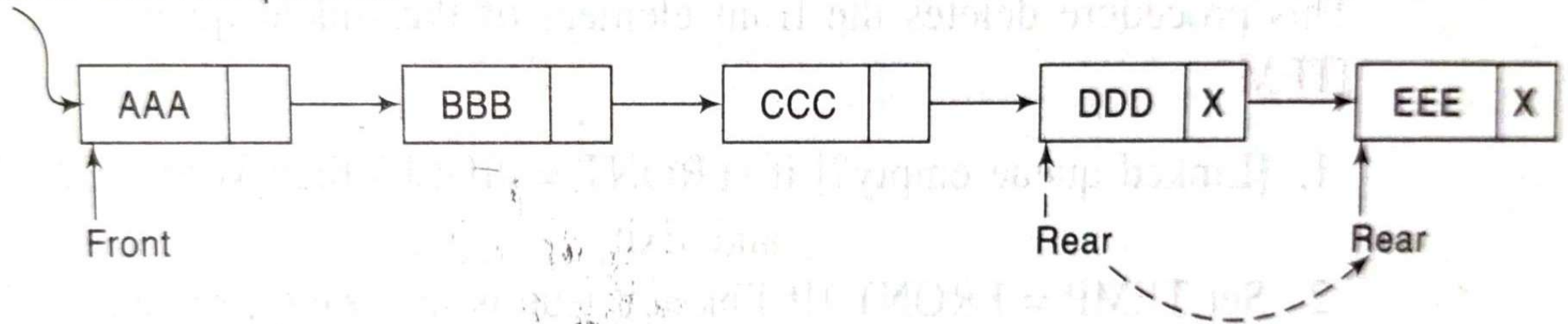


Fig. 6.23

Linked Representation of Queues

Delete from queue Q

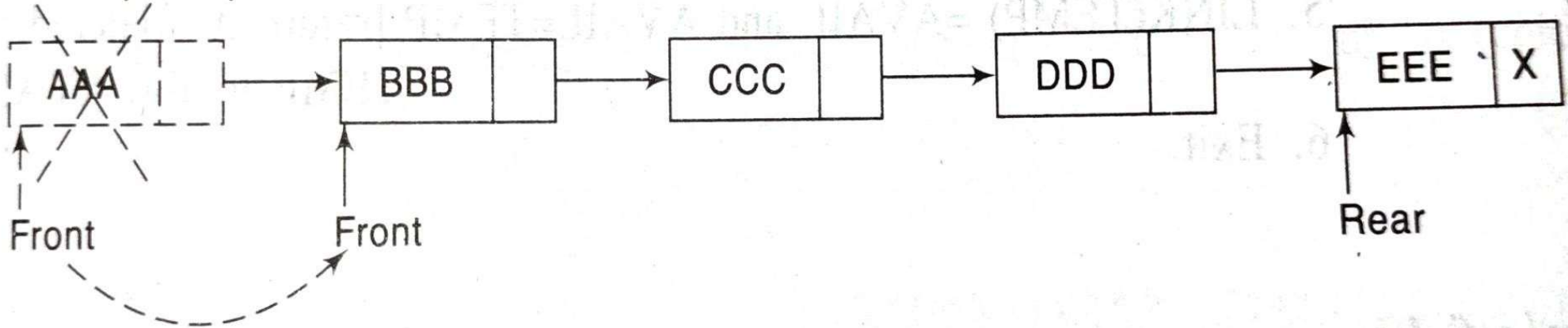


Fig. 6.24

Linked Representation of Queues

Procedure 6.15: LINKQ_INSERT(INFO, LINK, FRONT, REAR, AVAIL, ITEM)

This procedure inserts an ITEM into a linked queue

1. [Available space?] If AVAIL = NULL, then Write
OVERFLOW and Exit
2. [Remove first node from AVAIL list]
Set NEW := AVAIL and AVAIL := LINK[AVAIL]
3. Set INFO[NEW] := ITEM and LINK[NEW] = NULL
[Copies ITEM into new node]
4. If (FRONT = NULL) then FRONT = REAR = NEW
[If Q is empty then ITEM is the first element in the queue Q]
else set LINK[REAR] := NEW and REAR = NEW
[REAR points to the new node appended to
the end of the list]
5. Exit.

Linked Representation of Queues

Procedure 6.16: LINKQ_DELETE (INFO, LINK, FRONT, REAR, AVAIL, ITEM)

This procedure deletes the front element of the linked queue and stores it in ITEM

1. [Linked queue empty?] if (FRONT = NULL) then Write: UNDERFLOW and Exit
2. Set TEMP = FRONT [If linked queue is nonempty, remember FRONT in a temporary variable TEMP]
3. ITEM = INFO (TEMP)
4. FRONT = LINK (TEMP) [Reset FRONT to point to the next element in the queue]
5. LINK(TEMP) = AVAIL and AVAIL = TEMP [return the deleted node TEMP to the AVAIL list]
6. Exit.

Types of Queue

- Simple Queues
- Circular Queues
- Deque or Double-Ended Queues
- Priority Queues

Circular Queue

- In the case of an array implementation of a queue, it is possible that the insert function gives a signal stating that the even queue is full if the queue is partially free.
- To resolve this problem, the elements of the array should be shifted by one position to the left on every deletion that is made.
- This process is complex.
- An efficient another best way to overcome this issue is to assume that the array is circular, as shown in Fig. 6.26.
- A **circular queue** is a queue in which the element next to the last element is the first element.

Circular Queue

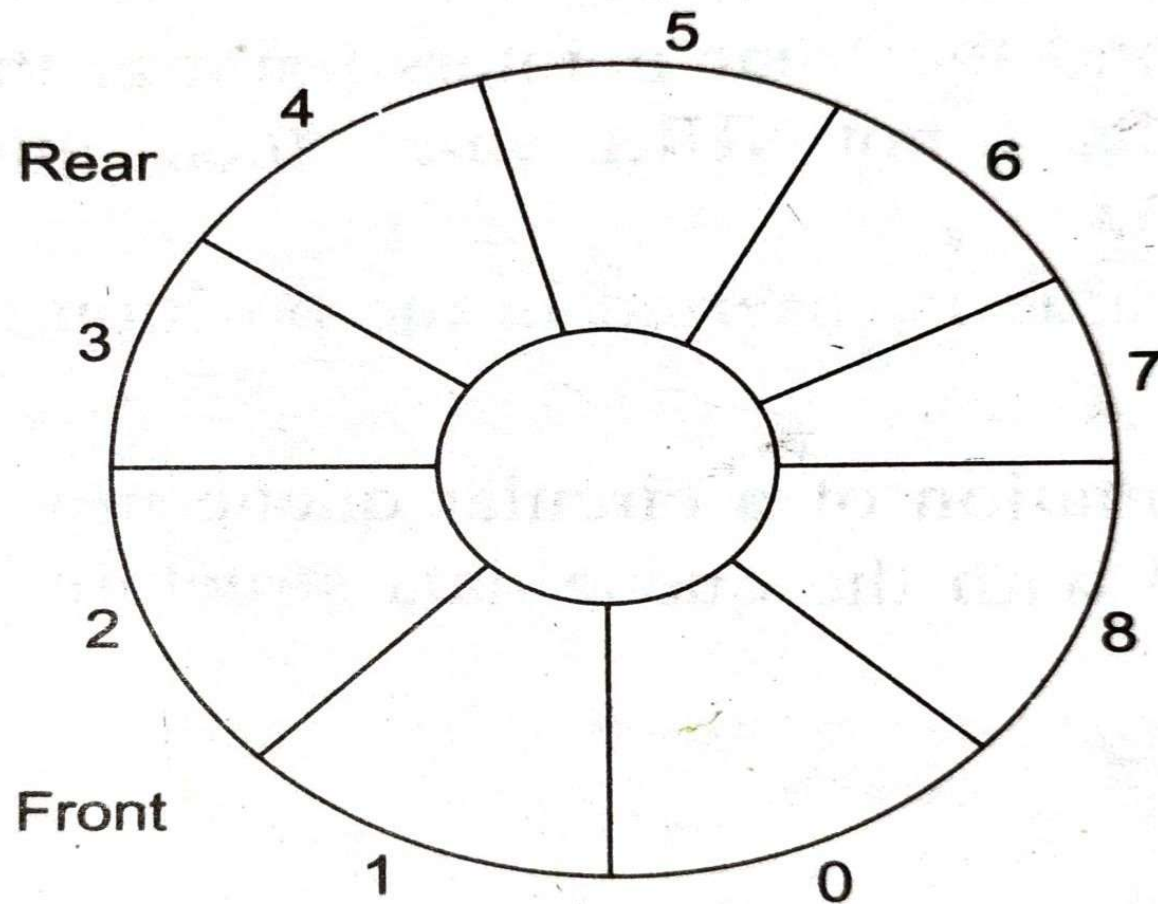
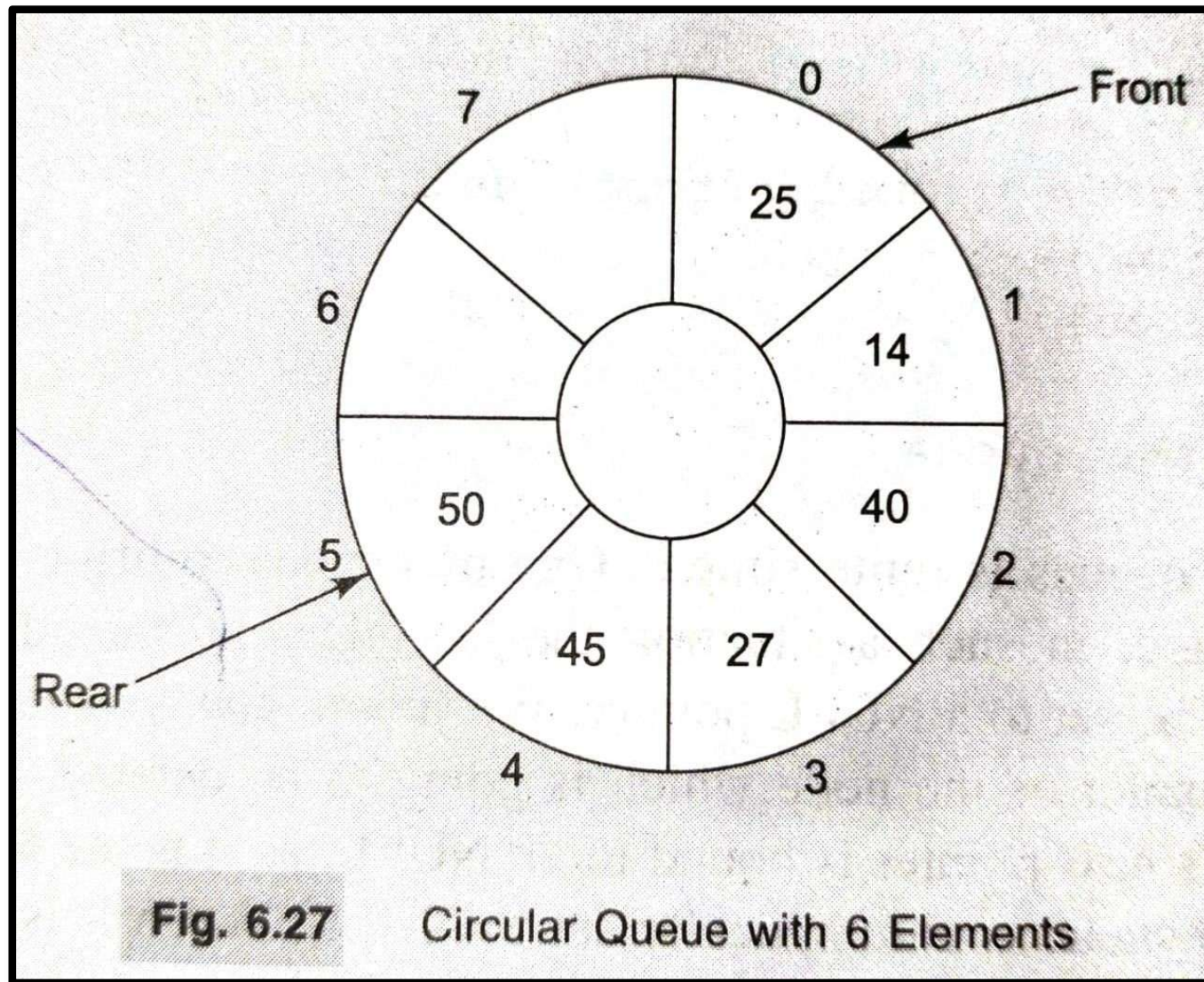


Fig. 6.26

Circular Queue

Circular Queue



Circular Queue

- In this example, the front and rear values are '0' and '5' respectively.
- On adding a element to the queue each time, the rear value increments by 1 until it reaches the upper limit of the queue (which is 7 in this case), and after this the process restarts again.
- In a similar fashion on deleting a queue element, the value of the front is incremented by 1 till the value becomes equal to the upper limit of the queue, after this the queue starts over again from 0.
- When we reach a point when the queue becomes full, the first element present in the queue will become the rear most element, this happens only when the front of the queue moves forward else it results in a queue overflow.

DEQUE

- A deque (pronounced either "deck" or "dequeue") is a linear list in which elements can be added or removed at either end but not in the middle. The term deque is a contraction of the name *double-ended-queue*.
- There are various ways of representing a deque in a computer.
- Unless it is otherwise stated or implied, we will assume our deque is maintained by a circular array DEQUE with pointers LEFT and RIGHT, which point to the two ends of the deque.
- We assume that the elements extend from the left end to the right end in the array.
- The term "circular" comes from the fact that we assume that DEQUE[1] comes after DEQUE[N] in the array.

DEQUE

- Figure 6.29 pictures two deques, each with 4 elements maintained in an array with $N = 8$ memory locations.
- The condition $LEFT = NULL$ will be used to indicate that a deque is empty.
- There are two variations of a deque-namely, an *input-restricted deque* and an *output-restricted deque* which are intermediate between a deque and a queue.
- An *input-restricted deque* is a deque which allows insertions at only one end of the list but allows deletions at both ends of the list. fig(a)
- An *output-restricted deque* is a deque which allows deletions at only one end of the list but allows insertions at both ends of the list. fig(b)

DEQUE

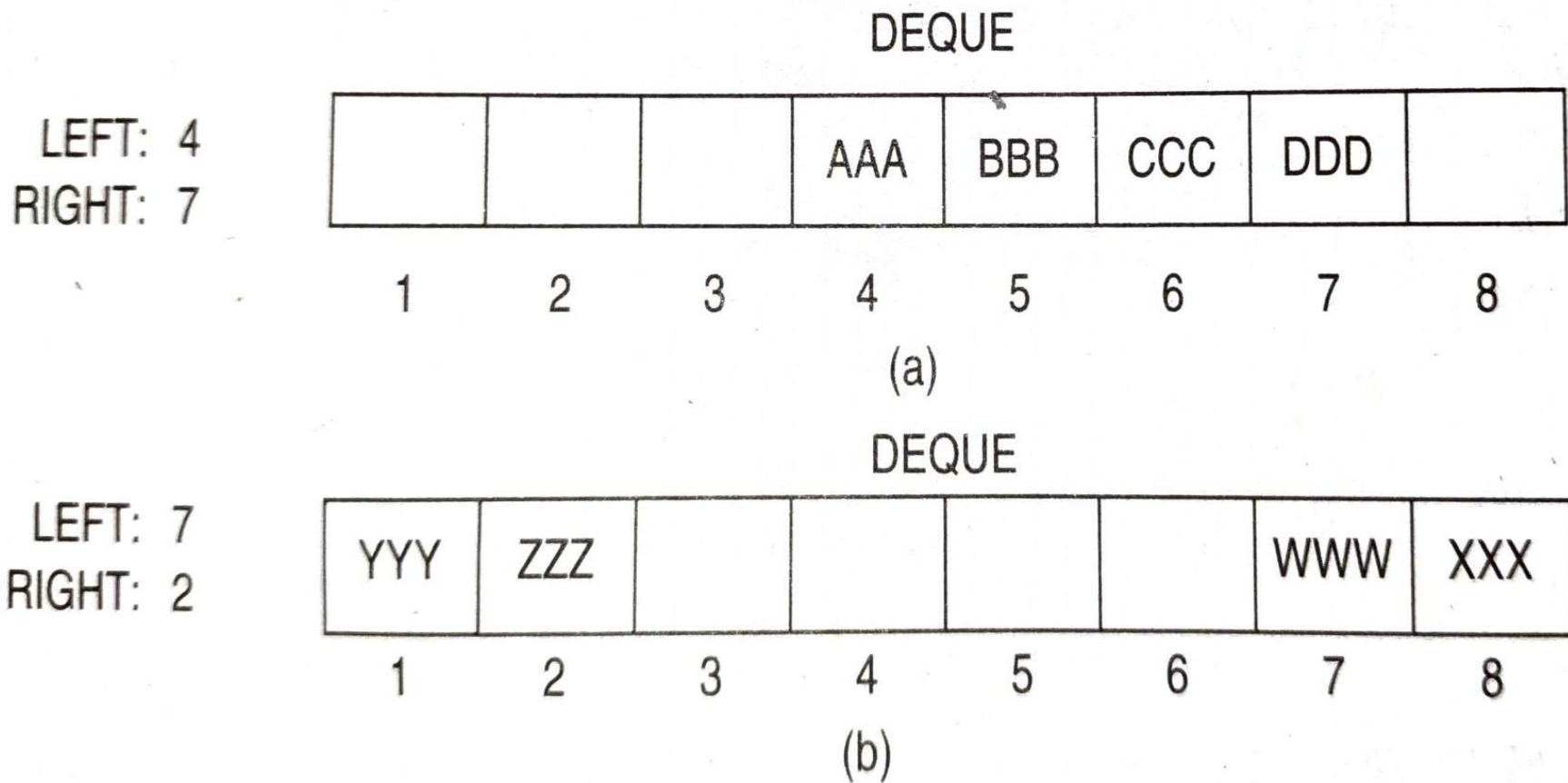


Fig. 6.29

DEQUE

- The procedures which insert and delete elements in deques and the variations on those procedures are given as supplementary problems.
- As with queues, a complication may arise
 - a) When there is overflow, that is, when an element is to be inserted into a deque which is already full.
 - b) When there is underflow, that is, when an element is to be deleted from a deque which is empty. The procedures must consider these possibilities.

Priority Queue

- A priority queue is a collection of elements such that each element has been assigned a priority.
- The order in which elements are deleted and processed comes from the following rules:
 1. An element of higher priority is processed before any element of lower priority.
 2. Two elements with the same priority are processed according to the order in which they were added to the queue.
- A prototype of a priority queue is a time sharing system: programs of high priority are processed first, and programs with the same priority form a standard queue.

One-way List Representation of PQ

- One-Way List Representation of a Priority Queue One way to maintain a priority queue in memory is by means of a one-way list, as follows:
 - a) Each node in the list will contain three items of information: an information field INFO, a priority number PRN and a link number LINK.
 - b) A node X precedes a node Y in the list
 1. When X has higher priority than Y or
 2. When both have the same priority but X was added to the list before Y.
- This means that the order in the one-way list corresponds to the order of the priority queue.
- Priority numbers will operate in the usual way: the lower the priority number, the higher the priority.

One-way List Representation of PQ

Example 6.15

Figure 6.30 shows a schematic diagram of a priority queue with 7 elements. The diagram does not tell us whether BBB was added to the list before or after DDD. On the other hand, the diagram does tell us that BBB was inserted before CCC, because BBB and CCC have the same priority number and BBB appears before CCC in the list. Figure 6.31 shows the way the priority queue may appear in memory using linear arrays INFO, PRN and LINK. (See Sec. 5.2.)

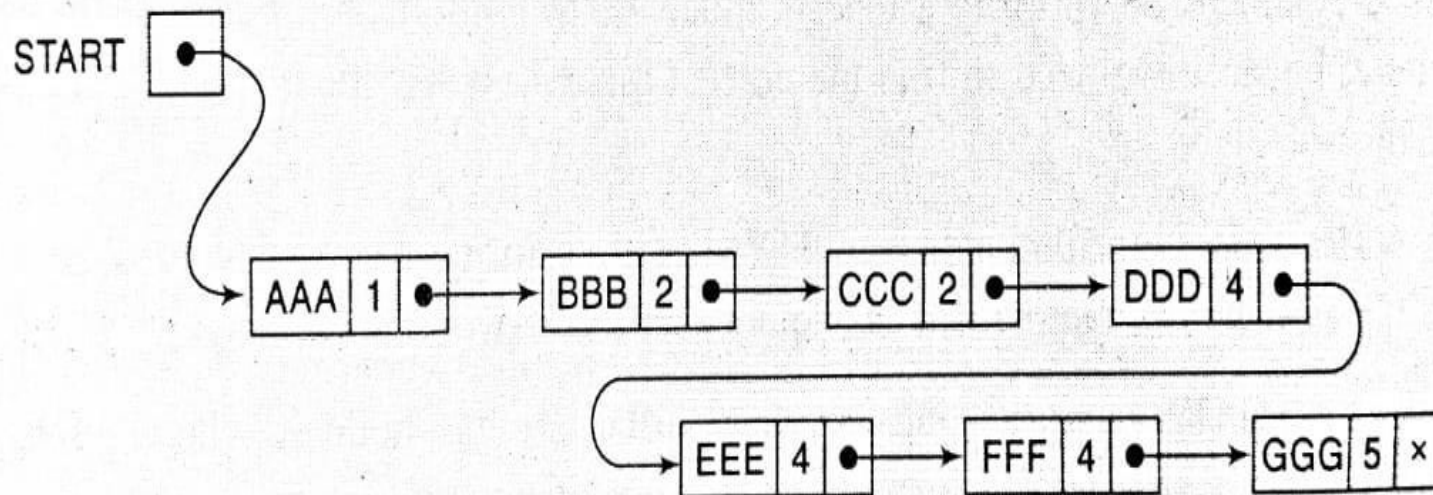


Fig. 6.30

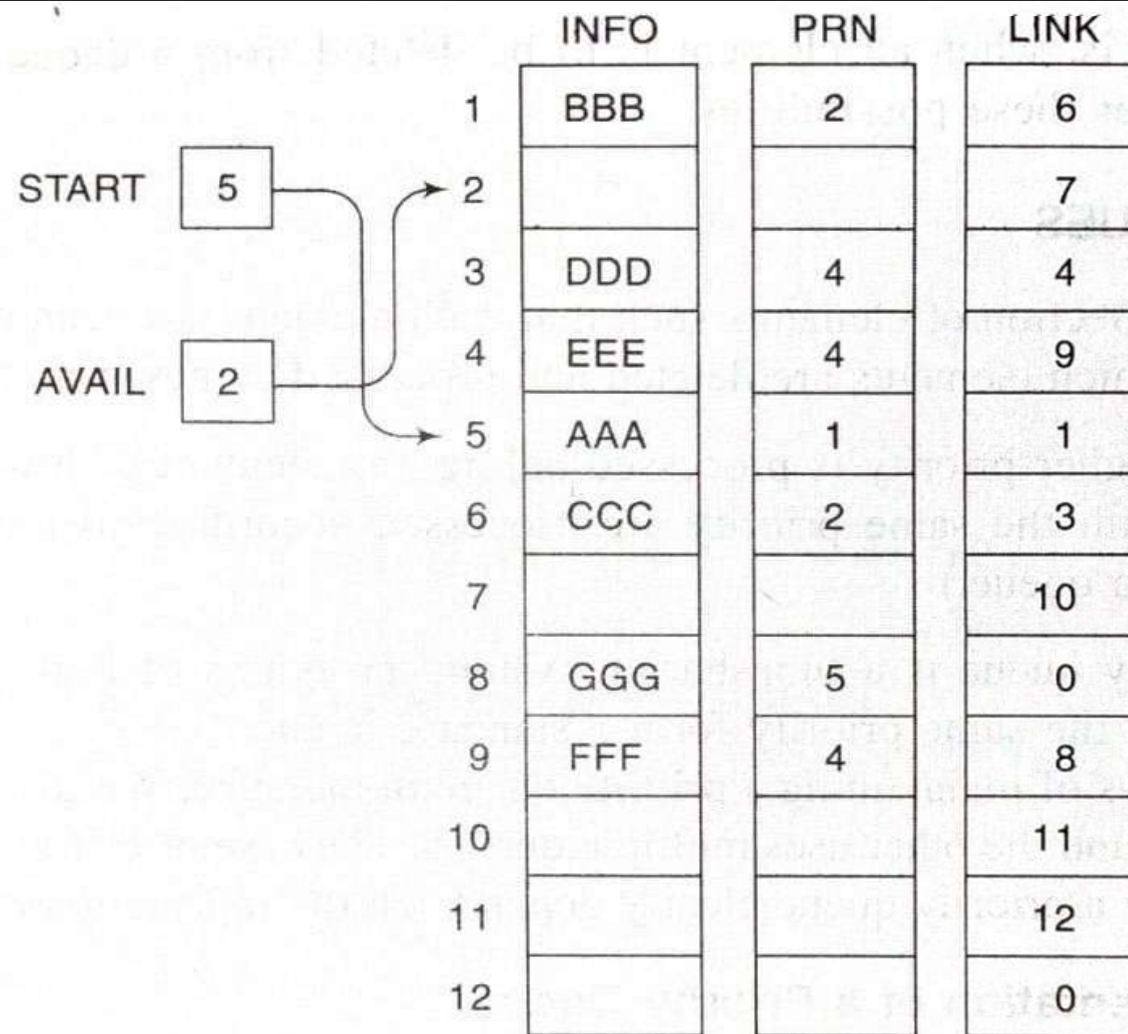


Fig. 6.31

The main property of the one-way list representation of a priority queue is that the element in the queue that should be processed first always appears at the beginning of the one-way list. Accordingly, it is a very simple matter to delete and process an element from our priority queue. The outline of the algorithm follows.

Types of Priority Queue

- Ascending Order Priority Queue
- Descending Order Priority Queue

Ascending order Priority Queue

- An ascending order priority queue gives the highest priority to the lower number in that queue.
- For example, you have six numbers in the priority queue that are 4, 8, 12, 45, 35, 20.
- Firstly, you will arrange these numbers in ascending order.
- The new list is as follows: 4, 8, 12, 20, 35, 45.
- In this list, 4 is the smallest number.
- Hence, the ascending order priority queue treats number 4 as the highest priority.

Ascending order Priority Queue

4	8	12	20	35	45
---	---	----	----	----	----

In the above table, 4 has the highest priority, and 45 has the lowest priority.

Descending order Priority Queue

- A descending order priority queue gives the highest priority to the highest number in that queue.
- For example, you have six numbers in the priority queue that are 4, 8, 12, 45, 35, 20.
- Firstly, you will arrange these numbers in ascending order.
- The new list is as follows: 45, 35, 20, 12, 8, 4. In this list, 45 is the highest number.
- Hence, the descending order priority queue treats number 45 as the highest priority.

Descending order Priority Queue

45	35	20	12	8	4
----	----	----	----	---	---

In the above table, 4 has the lowest priority, and 45 has the highest priority.